

Fluxograma completo do projeto

Este documento resume como o projeto está organizado hoje, como os dados circulam entre frontend, backend, banco e provedores externos, e onde a nova API de rota turística entra na arquitetura. A ideia é servir como mapa mental antes da próxima fase de refatoração e integração.

Resumo dos blocos do sistema

Frontend

Expo + React Native Web. Renderiza login, lugares, mapas e rotas.

Backend

Django + DRF. Expõe auth, catálogo de lugares, tickets, rotas e tour routes.

Banco

PostgreSQL + PostGIS. Sustenta places, auth e domínio persistido do app.

Provedores externos

Nominatim, OSRM e Overpass alimentam a API de rota turística sem banco.

1. Repositórios e responsabilidades

O projeto está separado em três repositórios. Isso ajuda a dividir bem a interface, a API e a documentação/modelagem.

Frontend

explora-plus-frontend

- App Expo / React Native com suporte web.
- Telas de login, cadastro, explorar, detalhes, perfil, tickets e rota.
- Componente de mapa baseado em Leaflet via WebView/iframe.

Backend

explora-plus-backend

- Django 5 + DRF + JWT + CORS.
- Apps: accounts, places, routes, tickets, tour_routes.
- places e auth estão mais prontos; routes/tickets ainda estão incompletos.

Dados

Postgres + PostGIS

- Persistência do catálogo de lugares e autenticação.
- Geo dados do app principal.
- Executa em container dedicado.

explora-plus-docs

- Modelagem, setup e artefatos de arquitetura.
- Este HTML serve como visão operacional complementar.

2. Fluxo macro do sistema

Este é o caminho principal que o usuário percorre hoje, do acesso à interface até o consumo de dados do backend e da API de rota turística.

Linha do fluxo 1

Usuário

Abre o app web/mobile, navega entre login, cadastro, explorar lugares e rota.

Frontend Expo

Chama a API do Django, renderiza listas, detalhes, mapa base e interações.

Django + DRF

Recebe requests, autentica quando necessário e orquestra respostas do domínio.

Banco ou provedores

Busca dados persistidos do app ou dados geográficos temporários externos.

Linha do fluxo 2

Resposta

JSON padronizado volta para o frontend com estado de auth, places ou rota turística.

Mapa

O frontend desenha polyline e marcadores no Leaflet em cima do OpenStreetMap.

UI final

Usuário enxerga cards, detalhe do lugar, histórico e futura rota inteligente.

3. Fluxo da API principal do app

Esta é a parte mais tradicional do sistema, com persistência em banco e autenticação JWT.

Autenticação

O frontend envia cadastro para `/api/auth/register/` ou login para `/api/auth/login/`. O backend cria o usuário e devolve tokens JWT quando o fluxo dá certo.

Catálogo de lugares

A tela de explorar consome `/api/places/` e a tela de detalhe consome `/api/places/`. Esses dados vêm do banco e já estão estruturados para o frontend.

Rotas e tickets do app principal

A arquitetura já reserva `/api/routes/` e `/api/tickets/`, mas hoje esses endpoints ainda são placeholders. Eles existem para o desenho do domínio, mas ainda não entregam a experiência final.

4. Fluxo detalhado da API de rota turística

A API POST `/api/tour-routes/` é o coração novo do projeto. Ela não usa banco e nem autenticação. Ela resolve origem e destino, calcula uma rota principal e procura lugares interessantes próximos do caminho.

Entrada do request

O cliente manda origin e destination, cada um com exatamente um entre address ou location.

Validação e normalização

O serializer do Django valida o formato e o planner decide se precisa geocodificar um texto ou se já pode seguir com coordenadas.

Geocoding com Nominatim

Se vier endereço, o backend chama o Nominatim para transformar esse texto em lat/lng e rótulo resolvido.

Rota base com OSRM

Com origem e destino resolvidos, o backend chama o OSRM público e recebe distância, duração e geometria completa da caminhada.

Busca de POIs com Overpass

O sistema abre um corredor ao redor da rota e busca cultura, parque e comida num raio máximo de 250 metros da linha principal.

- Cultura: museu, galeria, atração, historic, theatre, arts_centre.
- Parque: park, garden, viewpoint.
- Comida: restaurant, cafe.

Seleção inteligente dos POIs

Os candidatos são deduplicados por nome + proximidade, depois distribuídos ao longo da rota com alvo de 100 metros entre oportunidades de parada.

Montagem da resposta

O backend devolve dois blocos:

- route: origem, destino, distância, duração, polyline e places_to_pass.
- map: GeoJSON FeatureCollection com linha e marcadores.

Renderização no frontend

A ideia final é o frontend usar esses dados para desenhar a rota real no OpenStreetMap e apresentar os POIs como lista, pinos e pontos de parada.

5. Onde o projeto está desalinhado hoje

Frontend x backend

- A tela de rota do frontend ainda usa uma linha reta mockada entre origem e destino.
- O frontend continua apontando para `/api/routes/`, enquanto a API pronta está em `/api/tour-routes/`.

- Os tipos de marcador do frontend ainda não batem com os kinds do GeoJSON novo.

Estrutura do Django

- `tour_routes` está bem isolado, mas o app principal ainda tem routes placeholder.
- Na próxima refatoração, vocês provavelmente vão decidir se mantêm `tour_routes` separado ou se consolidam responsabilidades.
- O projeto também depende bastante do stack GIS no Docker, o que afeta a experiência local.

6. Próximos passos naturais

Integração imediata

- Fazer o frontend consumir POST `/api/tour-routes/`.
- Converter o GeoJSON em polyline e marcadores no mapa atual.
- Exibir `places_to_pass` como cards/lista clicável.

Refatoração estrutural

- Reorganizar apps e responsabilidades do Django.
- Definir a fronteira entre rota persistida do app e rota turística calculada em tempo real.
- Alinhar setup local para facilitar desenvolvimento e demonstração.

Gerado automaticamente a partir de fluxograma-projeto.html para consulta e compartilhamento offline.